Lecture_9

2025-10-22

- ## Example of RPART for titanic

```
library(rpart)
#This line loads the rpart package in R, which is used to build decision tree
models for classification and regression.

install.packages("rpart.plot")

library(rpart.plot)
#This line loads the rpart.plot package in R, which is used to visually plot
and display decision trees built with rpart.

url <- "https://intro-datascience.s3.us-east-
2.amazonaws.com/titanic.raw.Rdata"
download.file(url, "t.raw.Rdata")
load("t.raw.Rdata")
titanic <- titanic.raw
str(titanic)

## 'data.frame':    2201 obs. of  4 variables:
##  $ Class   : Factor w/ 4 levels "1st","2nd","3rd",..: 3 3 3 3 3 3 3 3 3 3
...
##  $ Sex     : Factor w/ 2 levels "Female","Male": 2 2 2 2 2 2 2 2 2 2 ...
##  $ Age     : Factor w/ 2 levels "Adult","Child": 2 2 2 2 2 2 2 2 2 2 ...
##  $ Survived: Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...
```

Explanation:

- The commands are for downloading and loading a dataset about Titanic from the internet into a software environment, like R, and then examining its structure.
- Here's a simple breakdown of the commands:
- url <- "...": You're telling the computer, where the data file is located and giving that address a short name (url).
- download.file(url, "t.raw.Rdata"): You're telling the computer to go to that web address and download the data file, saving it on your computer with the name "t.raw.Rdata".
- load("t.raw.Rdata"): You're telling the computer to open and read the data file you just downloaded and saved. This action creates a data object named titanic.raw in your working space.

- titanic <- titanic.raw: You're copying the data from the object named titanic.raw and giving it a shorter, easier-to-use name:
- str(titanic): You're asking the computer to tell you the structure of the titanic dataset. This command gives you a quick summary of what's inside the data.
- The output of str(titanic) provides a summary of the dataset. It tells you: What kind of data structure it is (e.g., a "data frame" or "table"). How many observations (rows) and variables (columns) are in the dataset. The name of each column (e.g., Class, Sex, Age, Survived). The type of data stored in each column (e.g., numbers, text, or factors which are categories).
- Since this is the "raw" Titanic dataset, the columns represent key information about the passengers and crew:

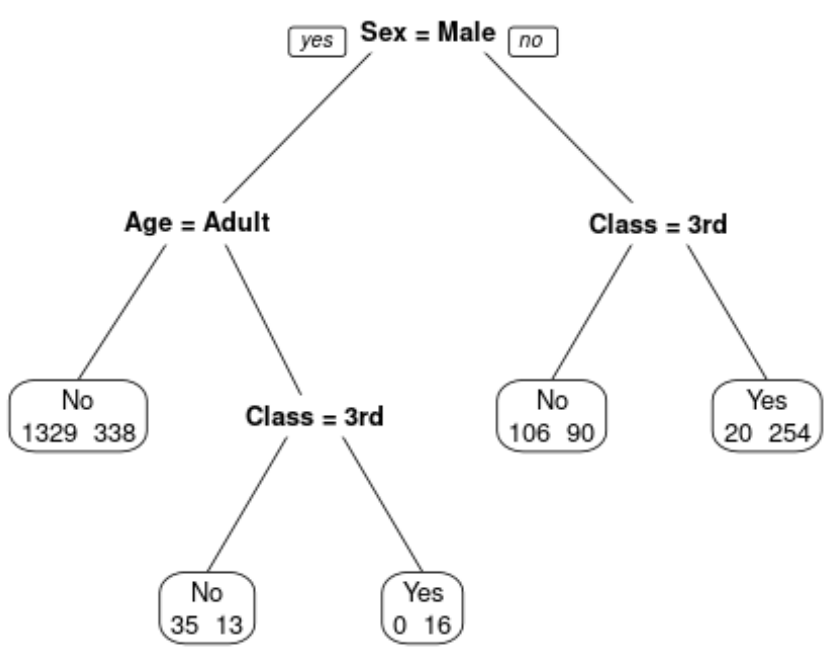
  Class: The type of ticket or position (e.g., 1st, 2nd, 3rd, Crew).

  Sex: The gender of the person (Male, Female).

  Age: Whether the person was an Adult or a Child.

  Survived: Whether the person lived or died (Yes, No).

```
#Build the model
cartTree <- rpart(Survived ~ ., data = titanic)
prp(cartTree, faclen = 0, cex = 0.8, extra = 1)
```



Explanation:

cartTree <- rpart(Survived ~ ., data = titanic)

Goal: You are building a Decision Tree model.

- rpart(...): This is the function that builds the tree (it stands for Recursive Partitioning).
- Survived ~ .: This is the formula for the model: Survived is what you want to predict (Did the person live or die?). ~ . means you want to use all other columns in the dataset (Class, Sex, Age, etc.) to make that prediction.
- data = titanic: You are telling the function to use the titanic dataset you loaded previously.
- cartTree <- ...: You are saving the completed Decision Tree model into a new object called cartTree.

prp(cartTree, faclen = 0, cex = 0.8, extra = 1)

Goal: You are visually displaying the Decision Tree you just built.

- prp(...): This is the function used to plot the tree in a nice, readable way (it stands for Pretty RPart Plot).
- cartTree: This is the model you are telling the function to draw.
- faclen = 0, cex = 0.8, extra = 1: These are just formatting options to make the plot look good:They control things like how short the category names are (faclen = 0), the size of the text (cex = 0.8), and what extra information is shown on the nodes of the tree (extra = 1).

Diagram Description: What the Numbers Mean:

- In a classification tree, the numbers in the final boxes (the "leaves") represent how well the model performed for the two possible outcomes:
- Left Pointing Leaves: First Number: The count of cases that were correctly predicted as being in the first category (e.g., "Not Survived").
- Second Number: The count of cases that were incorrectly predicted (they were actually in the second category, e.g., "Survived," but the tree predicted the first).
- Right Pointing Leaves (Vice Versa): The meaning of the two numbers is swapped compared to the left-pointing leaves.
- First Number: The count of cases that were incorrectly predicted.
- Second Number: The count of cases that were correctly predicted as being in the second category (e.g., "Survived").

Preeti Jagadev

```
cartTree
```

```
## n= 2201
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
##
##  1) root 2201 711 No (0.6769650 0.3230350)
##    2) Sex=Male 1731 367 No (0.7879838 0.2120162)
##      4) Age=Adult 1667 338 No (0.7972406 0.2027594) *
##      5) Age=Child 64  29 No (0.5468750 0.4531250)
##       10) Class=3rd 48  13 No (0.7291667 0.2708333) *
##       11) Class=1st,2nd 16   0 Yes (0.0000000 1.0000000) *
##    3) Sex=Female 470 126 Yes (0.2680851 0.7319149)
##      6) Class=3rd 196  90 No (0.5408163 0.4591837) *
##      7) Class=1st,2nd,Crew 274  20 Yes (0.0729927 0.9270073) *
```

Predicting values using the Model:

```
predictValues <- predict(cartTree, newdata=titanic)
titanic[1,]
```

```
##   Class  Sex   Age Survived
## 1   3rd Male Child       No
```

Explanation:

predictValues <- predict(cartTree, newdata=titanic)

Goal: Use the tree model you built to make a prediction for every person in the original dataset.

- predict(cartTree, ...): You're telling the computer to use your saved Decision Tree model (cartTree).
- newdata=titanic: You're telling the computer to make predictions for all the data rows in the titanic dataset.
- predictValues <- ...: You're saving all those predictions (the likelihood of survival for each person) into a new object called predictValues.

titanic1,

Goal: Look at the first entry in the dataset.

- titanic: This is your main dataset containing all the information about passengers and crew.
- 1 refers to the first row (the first observation/person) , followed by nothing means you want all the columns for that row.

Preeti Jagade

```
predictValues[1]
```

```
## [1] 0.7291667
```

```
predictValues <- predict(cartTree, newdata=titanic, type = "class")
titanic[1,]
```

```
##   Class  Sex   Age Survived
## 1   3rd Male Child       No
```

Explanation:

These two lines of code use the **Decision Tree model** to make a prediction for every person in the dataset and then look at the details of the first person.

predictValues <- predict(cartTree, newdata=titanic, type = "class")
Goal: You are using the saved Decision Tree (cartTree) to make a definitive prediction (a label like "Yes" or "No") for everyone in your dataset.

- predict(...): The instruction to use your model to make predictions.
- cartTree: The specific set of rules (the model) you want to use.
- newdata=titanic: You're asking for a prediction for every person in the entire titanic dataset.
- type = "class": This is the key difference! It tells the function not to output a probability (a number between 0 and 1), but to output the final classification category (a "Yes" or "No" prediction for survival).
- predictValues <- : All these "Yes" or "No" predictions are saved into a new list called predictValues.

Simple Summary: "Use the model to predict a final Yes (Survived) or No (Died) answer for every person in the data, and save those answers."

```
predictValues[1]
```

```
##  1
## No
## Levels: No Yes
```

What was the Accuracy

```
actualSurvived <- as.factor(titanic$Survived == "Yes")
confMatrix <- table(predictValues,actualSurvived)
confMatrix
```

```
##               actualSurvived
## predictValues FALSE TRUE
```

Geetijagadev

```
##            No   1470  441
##            Yes    20  270
```

These lines of code are preparing the data and then creating a Confusion Matrix to check how accurate your Decision Tree model was at predicting survival.

actualSurvived <- as.factor(titanic$Survived == "Yes")

Goal: Create a simple list of the true outcome (what actually happened) for every person in the data.

- titanic$Survived == "Yes": This checks the Survived column in your data and creates a list of TRUE (if they survived) or FALSE (if they died) for every single person.
- as.factor(...): This converts that list into a formal categorical variable, which is necessary for the next step.
- actualSurvived <- ...: You are saving this final list of true outcomes into a new object named actualSurvived.

Simple Summary: Make a list called actualSurvived where every person is labeled simply as TRUE (lived) or FALSE (died).

confMatrix <- table(predictValues, actualSurvived)

Goal: Compare the model's predictions against the actual outcomes.

- table(...): This is the function that creates a crosstabulation (a two-way table).
- predictValues: This is the list of outcomes the model predicted (your Yes/No or TRUE/FALSE predictions from the previous steps).
- actualSurvived: This is the list of true outcomes (what really happened) you just created.
- confMatrix <- ...: The resulting table is saved into an object named confMatrix (short for Confusion Matrix).
- The numbers along the diagonal (1470 and 270) are the correct predictions.The numbers off the diagonal (441 and 20) are the mistakes (the "confusion"). This matrix is the basis for calculating your model's accuracy.

```
accuracy <- 1 - (sum(confMatrix)  -  sum(diag(confMatrix)))/ sum(confMatrix)
# This code is calculating the Accuracy of your Decision Tree model using the
numbers from the Confusion Matrix (confMatrix).
accuracy
```

```
## [1] 0.7905498
```

Preeti Jagadev

Using Caret to create Training and Test sets

```
install.packages("caret")
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'
## (as 'lib' is unspecified)

library(caret)

## Loading required package: ggplot2

## Loading required package: lattice

# makes the sampling predictable
set.seed(110)

# Randomly sample elements to go into a training data set
trainList <- createDataPartition(y=titanic$Survived,p=.80,list=FALSE)
```

Explanation:

This command splits your Titanic dataset into two groups: an 80% training group and a 20% testing group, making sure both groups have a similar proportion of survivors and non-survivors.

The code uses the createDataPartition function (from the caret package in R) to select which rows belong in the training set.

- y = titanic$Survived: This is the most crucial part. You are telling the function to look at the Survived column when splitting the data. This ensures the resulting 80% and 20% groups both have a roughly equal proportion of people who survived and people who died (a process called stratification).
- p = .80: This specifies the size of the first group. It means you want the training set to contain 80% of the total observations. The remaining 20% will be used for the testing set.
- list = FALSE: This tells the function to return a simple vector of row numbers (indices) that belong to the 80% training group, making it easy to use these numbers to select the actual data rows later.

The final object, trainList, is not the data itself, but a list of numbers indicating which rows from the original titanic table should be used to train your model.

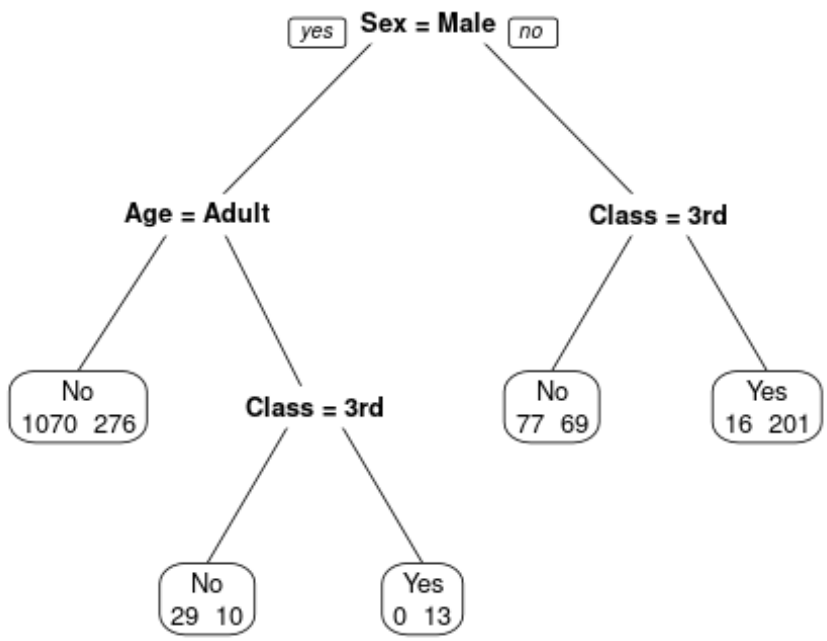
```
# Include all of those elements in the training set
trainSet <- titanic[trainList,]

# Construct test set from everything that didn't go into the training
testSet <- titanic[-trainList,]

head(trainList)
```

Preeti Jagade

```
##      Resample1
## [1,]         1
## [2,]         2
## [3,]         4
## [4,]         5
## [5,]         6
## [6,]         7
```

Use the trainSet:

```
cartTree <- rpart(Survived ~ ., data = trainSet)
prp(cartTree, faclen = 0, cex = 0.8, extra = 1)
```



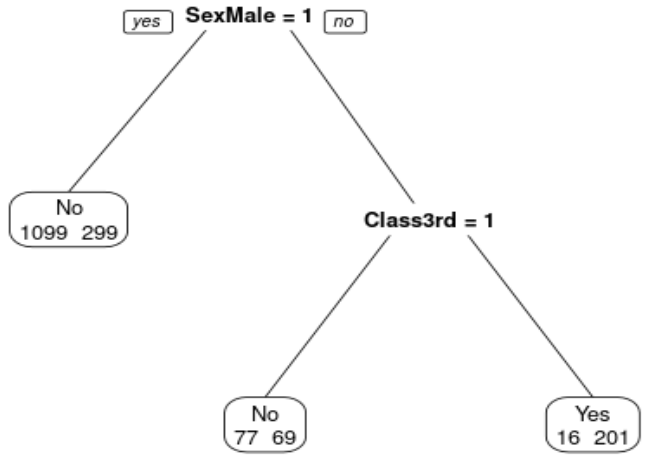
Evaluate using the testSet

```
predictValues <- predict(cartTree,newdata=testSet, type = "class")
confusion <- confusionMatrix(predictValues, testSet$Survived)
confusion$overall[1]
```

```
##  Accuracy
## 0.7954545
```

Using Caret to build a model

```
cartTree1 <- train(Survived ~ ., data=trainSet,  method="rpart")
prp(cartTree1$finalModel, faclen = 0, cex = 0.8, extra = 1)
```



Explanation:

This code builds and displays a Decision Tree model but uses a more advanced function (train) that automatically helps you find the best version of the tree.

cartTree1 <- train(Survived ~ ., data=trainSet, method="rpart")
Goal: To build the best possible Decision Tree model and save it.

- train(...): This is a powerful function (from the caret package) that doesn't just build a model; it automatically tunes or tests different versions of the model (called cross-validation) to see which parameters give the best performance.
- Survived ~ .: The standard formula, meaning "predict Survived using all other variables."
- data = trainSet: It uses only your designated training data.
- method = "rpart": This tells the train function that the specific type of model you want it to optimize is a Decision Tree (the rpart method).
- cartTree1 <- ...: The entire optimized training process and the resulting best model are saved into the new object cartTree1.

prp(cartTree1$finalModel, faclen = 0, cex = 0.8, extra = 1)
Goal: To draw the best tree that the train function found.

- prp(...): The standard function to plot the tree.
- cartTree1$finalModel: Because cartTree1 is a complex object containing all the test results, you must specifically pull out the finalModel (the single best tree determined by the tuning process) for plotting.

Preeti Jagadev

- Formatting Arguments: As before, arguments like faclen=0 and cex=0.8 are just settings to make the plot easy to read.

```
cartTree1
```

```
## CART
##
## 1761 samples
##    3 predictor
##    2 classes: 'No', 'Yes'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 1761, 1761, 1761, 1761, 1761, 1761, ...
## Resampling results across tuning parameters:
##
##   cp          Accuracy   Kappa
##   0.01142355  0.7807280  0.4331489
##   0.01405975  0.7767017  0.4237167
##   0.31107206  0.7276983  0.2230516
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.01142355.
```

```
cartTree1$finalModel
```

```
## n= 1761
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
##
## 1) root 1761 569 No (0.67688813 0.32311187)
##   2) SexMale>=0.5 1398 299 No (0.78612303 0.21387697) *
##   3) SexMale< 0.5 363  93 Yes (0.25619835 0.74380165)
##     6) Class3rd>=0.5 146  69 No (0.52739726 0.47260274) *
##     7) Class3rd< 0.5 217  16 Yes (0.07373272 0.92626728) *
```

Exploring the model build via Caret

```
predictValues <- predict(cartTree1,newdata=testSet, type = "raw")
#type = "raw": This tells the function to output the final classification
label (e.g., "Yes" or "No" for Survived), rather than just a probability.
confusion <- confusionMatrix(predictValues, testSet$Survived)
confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No Yes
##        No  294  89
```

```
##        Yes   4  53
##
##                Accuracy : 0.7886
##                  95% CI : (0.7475, 0.8259)
##     No Information Rate : 0.6773
##     P-Value [Acc > NIR] : 1.476e-07
##
##                   Kappa : 0.4267
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.9866
##             Specificity : 0.3732
##          Pos Pred Value : 0.7676
##          Neg Pred Value : 0.9298
##              Prevalence : 0.6773
##          Detection Rate : 0.6682
##    Detection Prevalence : 0.8705
##       Balanced Accuracy : 0.6799
##
##        'Positive' Class : No
##
```

Exploring Variable Importance

```r
varImp(cartTree)
```

```
##        Overall
## Age    14.3582
## Class 106.1944
## Sex   161.8500
```

Explanation:

- varImp(...): This function calculates the Variable Importance, or how much each input variable (like Sex, Class, or Age) contributed to the model's accuracy.
- cartTree: This is the specific Decision Tree model you built that the function will analyze.

```r
varImp(cartTree1)
```

```
## rpart variable importance
##
##           Overall
## SexMale   100.000
## Class3rd   24.107
## ClassCrew   3.650
## Class2nd    1.233
## AgeChild    0.000
```

The German Credit Database:

```
# Just grab a subset of the data for the demo
data("GermanCredit")
subCredit <- GermanCredit[,1:10]

# makes the sampling predictable
set.seed(111)

# Randomly sample elements to go into a training data set
trainList <-
          createDataPartition(y=subCredit$Class,p=.80,list=FALSE)

# create test and train datasets
trainSet <- subCredit[trainList,]
testSet <- subCredit[-trainList,]
```

Train Using a treebag model

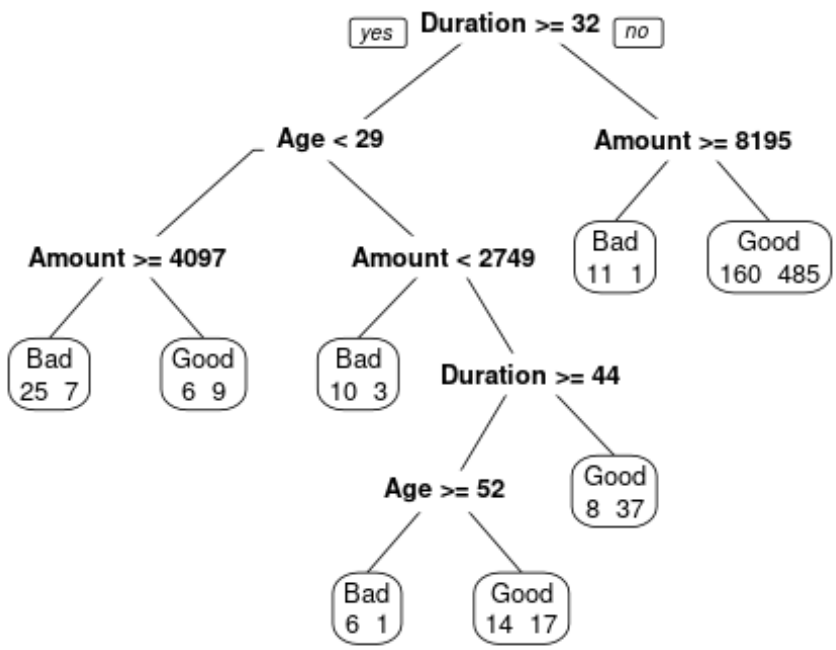
```
fit1 <- train(Class ~ ., data=trainSet, method="treebag",
preProc=c("center","scale"))
```

Explanation:

- Class ~ : The model should predict the Class based on all other variables (.) in the data.
- data=trainSet: Use the data stored in the trainSet to learn the patterns.
- method="treebag" : Use the 'Bagged Classification Trees' technique to build the model."
- preProc=c("center","scale"): Before training, prepare the data by centering (making the average zero) and scaling (standardizing the spread) all the numeric variables. This helps the algorithm work better.

```
# Interpret Results: Variable Importance
varImp(fit1)

## treebag variable importance
##
##                            Overall
## Amount                     100.000
## Age                         76.137
## Duration                    57.673
## ResidenceDuration           34.657
## InstallmentRatePercentage   33.085
## NumberExistingCredits       21.171
## Telephone                   16.459
## NumberPeopleMaintenance      7.876
## ForeignWorker                0.000
```

```r
cartTree <- rpart(Class ~ Amount + Age + Duration, data = trainSet,
method="class")
prp(cartTree, faclen = 0, cex = 0.8, extra = 1)
```



This code snippet performs two main actions:

1. **Creating the Model: (cartTree <- rpart(...)):** This line builds a predictive model using a technique called Classification and Regression Trees (CART).

- cartTree <- : Create a model and name it cartTree.
- rpart: Start the process to build a decision tree (the CART method).
- Class ~ Amount + Age + Duration,: The model should predict the Class (the outcome, like 'Good Risk' or 'Bad Risk') using three specific pieces of information: Amount, Age, and Duration."
- data = trainSet,: Use the data stored in trainSet to learn the patterns.
- method = "class": Build a Classification Tree (meaning the model predicts a category or group, like 'Yes'/'No' or 'A'/'B').

2. **Displaying the Model (prp(cartTree, ...))**

This line visualizes the Decision Tree you just created so you can easily understand its logic.

- prp(cartTree,: Plot the tree (a simplified, easy-to-read version) stored in the cartTree object.
- faclen = 0,: Don't abbreviate the names of the factor/class levels.
- cex = 0.8,: Make the text size in the plot a little smaller (80% of normal) so it fits better.
- extra = 1: "Add some extra information to the plot's nodes, specifically showing the counts/totals for each category."

```
predOut <- predict(fit1, newdata=testSet)
confusion <- confusionMatrix(predOut, testSet$Class)
confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad Good
##       Bad   24   20
##       Good  36  120
##
##                Accuracy : 0.72
##                  95% CI : (0.6523, 0.781)
##     No Information Rate : 0.7
##     P-Value [Acc > NIR] : 0.29717
##
##                   Kappa : 0.2784
##
##  Mcnemar's Test P-Value : 0.04502
##
##             Sensitivity : 0.4000
##             Specificity : 0.8571
##          Pos Pred Value : 0.5455
##          Neg Pred Value : 0.7692
##              Prevalence : 0.3000
##          Detection Rate : 0.1200
##    Detection Prevalence : 0.2200
##       Balanced Accuracy : 0.6286
##
##        'Positive' Class : Bad
##
```

```
confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad Good
##       Bad   24   20
##       Good  36  120
```

```
##
##                Accuracy : 0.72
##                  95% CI : (0.6523, 0.781)
##     No Information Rate : 0.7
##     P-Value [Acc > NIR] : 0.29717
##
##                   Kappa : 0.2784
##
##  Mcnemar's Test P-Value : 0.04502
##
##             Sensitivity : 0.4000
##             Specificity : 0.8571
##          Pos Pred Value : 0.5455
##          Neg Pred Value : 0.7692
##              Prevalence : 0.3000
##          Detection Rate : 0.1200
##    Detection Prevalence : 0.2200
##       Balanced Accuracy : 0.6286
##
##        'Positive' Class : Bad
##
```

Use SVM to Explore the Credit Data

```
data("GermanCredit")
subCredit <- GermanCredit[,1:10]

# makes the sampling predictable
set.seed(111)

# Randomly sample elements to go into a training data set
trainList <- createDataPartition(y=subCredit$Class,p=.80,list=FALSE)
trainSet <- subCredit[trainList,]
testSet <- subCredit[-trainList,]

fit2 <- train(Class ~ ., data=trainSet, method="svmRadial",
preProc=c("center","scale"))
```

Explanation:

This line of code instructs a computer program to build a prediction model using a powerful, non-linear technique.

- fit2 <-: Create a model and name it fit2.
- train: Start the training process to build the model.
- Class ~ .: The model should predict the outcome Class based on all other variables (.) in the data.

- data=trainSet: Use the data stored in trainSet (your known training examples) to teach the model the patterns.
- method= svmRadial: Use the Support Vector Machine with a Radial Basis Function Kernel (RBF-SVM) technique to build the model.

This is a sophisticated method that finds the best possible curved boundary to separate different categories in your data. It's especially good for problems where a simple straight line just won't work. It uses the Radial function to map the data into a higher dimension, making complex separation possible.

- preProc=c("center","scale"): Before training, prepare the data by centering (making the average zero) and scaling (standardizing the spread) all the numeric variables.

Why this is important: SVM models like this one are very sensitive to the scale of the data. This step ensures that all variables contribute equally to the model, preventing variables with large numeric values (like Amount) from dominating the prediction simply because their numbers are bigger than others (like Age).

```
predOut <- predict(fit2, newdata=testSet)

confusion <- confusionMatrix(predOut, testSet$Class)
confusion

## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad Good
##       Bad   10    6
##       Good  50  134
##
##                Accuracy : 0.72
##                  95% CI : (0.6523, 0.781)
##     No Information Rate : 0.7
##     P-Value [Acc > NIR] : 0.2972
##
##                   Kappa : 0.1566
##
##  Mcnemar's Test P-Value : 9.132e-09
##
##             Sensitivity : 0.1667
##             Specificity : 0.9571
##          Pos Pred Value : 0.6250
##          Neg Pred Value : 0.7283
##              Prevalence : 0.3000
##          Detection Rate : 0.0500
##    Detection Prevalence : 0.0800
##       Balanced Accuracy : 0.5619
##
```

```
##        'Positive' Class : Bad
##
```

```
fit2
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 800 samples
##   9 predictor
##   2 classes: 'Bad', 'Good'
##
## Pre-processing: centered (9), scaled (9)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 800, 800, 800, 800, 800, 800, ...
## Resampling results across tuning parameters:
##
##   C     Accuracy   Kappa
##   0.25  0.6968530  0.007985816
##   0.50  0.7001655  0.076417246
##   1.00  0.7029363  0.122703817
##
## Tuning parameter 'sigma' was held constant at a value of 0.1041149
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were sigma = 0.1041149 and C = 1.
```

Preeti Jad